ACADEMIC MAJOR PROJECT REPORT & ARCHITECTURAL DOSSIER

GeoScout: Natural Language Interpretation & Precision Telemetry for Earth Observation Imagery

A Unified Three-Stage Framework Integrating Vision-Language Reasoning (Qwen3-VL), Zero-Shot Concept Segmentation (SAM 3), and Physical Geospatial Measurement

---

Candidate Author: Dhruv Sarin
Project Track: Computer Science & Engineering — Capstone Major Project
Base Repository: GeoNLI (Inter-IIT Tech Meet / ISRO Problem Statement 65)
Evaluation Reference: Original GeoNLI Academic Paper (Team 65, attached as reference)
Audit Date: October 2026
Implementation Audit: 100% Code-Backed Ledger (All files verified in workspace)

Table of Contents

1. The Project in 60 Seconds — Executive summary, intended users, and 3-sentence elevator pitch

2. The Problem Being Solved — Why satellite imagery is locked away from ordinary users and real units

3. Why Not Just Use a Chatbot or LLM Directly? — Honest analysis of LLM hallucinations vs. specialized pipelines

4. How It Works: The Architecture in Simple Language — The 'Team of Specialists' analogy and end-to-end dataflow

5. Why This Approach and Not Others — Technical justification for 3 stages, fine-tuning (GSPO, LoRA), and OpenCV

6. What is Original GeoNLI Work and What is New in My Project — Transparent ledger separating Team 65 research from my engineering

7. Features, One by One — Exhaustive tour of all 10 application features, inner workings, and limits

8. The Agriculture and Flood Use Case, Explained Step by Step — Worked Punjab SAR flood case study, Otsu thresholding, and telemetry

9. Live Mode vs. Demo Mode — Dual-path architecture, academic integrity banners, and GPU independence

10. Results and Validation — VRSBench benchmarks, grounding IoU, and Sen1Floods11 disclosure

11. Limitations and Honest Weaknesses — GPU dependencies, single-stream concurrency, optical vs. SAR caveats

12. How to Run It — Prerequisites, laptop vs. Colab setup, tunnel bridge, common error fixes

13. Future Work — Realistic next steps: edge quantization, Redis queuing, and automated GSD

14. Presentation Help — 10-minute talk outline, 2-minute pitch, and 25 professor Q&As

15. Glossary — Plain-language definitions of all technical acronyms and concepts

16. Appendix — Repository file map, API endpoint specification, and academic references

1. The Project in 60 Seconds

GeoScout is an intelligent geospatial analysis system that lets anyone upload a satellite or aerial image and ask questions in plain English—such as 'How many circular irrigation fields are here?' or 'What is the total flooded cropland area in hectares?'—and receive instant, pixel-accurate answers, visual boundary maps, and real-world physical measurements.

Built upon the foundational research of the GeoNLI project (developed for the Inter-IIT Tech Meet under an Indian Space Research Organisation / ISRO problem statement by Team 65), GeoScout bridges the massive gap between raw, complex Earth Observation satellite data and human decision-makers. Instead of requiring a team of GIS (Geographic Information Systems) professionals working for days in complicated desktop software, GeoScout delivers answers in five seconds through a modern web browser.

1.1 Who Is It For?

- **Agricultural Officers & Farmers:** Who need to verify crop parcel boundaries, calculate exact cultivated acreage, and assess flood damage for crop insurance claims without manual field surveys.
- **Disaster Response & Revenue Teams:** Who must rapidly quantify submerged land after major monsoons or river breaches, producing official bilingual relief audit reports in Hindi and English.
- **Environmental & Urban Planners:** Who need to monitor forest encroachment, industrial land use, water bodies, and infrastructure without learning specialized remote sensing code.

1.2 What Does the User See on Screen?

When a user opens GeoScout in their web browser, they are presented with a clean, dual-pane geospatial workspace:

- **On the Left:** An interactive GIS canvas displaying the high-resolution satellite scene, equipped with fluid pan, wheel zoom (from 1x to 8x), an opacity slider, a split-screen before-and-after comparison slider, and color-coded vector polygon overlays highlighting every detected object or field parcel.
- **On the Right:** An AI conversational intelligence stream. When the user asks a question, this panel displays the vision model's chain-of-thought reasoning monologue, the exact numerical answer, a telemetry card detailing physical hectares and acres, and one-click export buttons for GeoJSON, ESRI Shapefiles, and printable audit reports.

> **KEY IDEA:** GeoScout transforms satellite imagery from an inaccessible grid of colored pixels into actionable physical intelligence (hectares, acres, counts, and vector boundaries) through simple natural language conversation.

> **SAY THIS TO YOUR PROFESSOR:** "Professor, GeoScout is a full-stack Earth Observation intelligence platform. It combines a fine-tuned vision-language model for reasoning, a Segment Anything foundation model for sub-pixel boundary tracing, and deterministic geometry to convert satellite pixels into verified real-world hectares. I took the raw Inter-IIT GeoNLI research pipeline and transformed it into a deployable, multi-modal decision-support system featuring live radar flood analytics, bilingual disaster auditing, and GIS vector

> exports."

2. The Problem Being Solved

Satellites orbiting the Earth capture millions of square kilometers of high-resolution imagery every single day. Government agencies like ISRO, NASA, and the European Space Agency make enormous volumes of optical and radar imagery publicly available. Yet, despite this wealth of data, very little of it can be used directly by the people who need it most.

2.1 Why Satellite Imagery Is So Hard to Use Today

- **Requirement for Highly Trained Specialists:** Interpreting overhead imagery requires specialized training in photogrammetry, remote sensing, and spectral bands. Ordinary administrators cannot look at an optical or radar scene and reliably isolate complex patterns.
- **Clunky, Complex Desktop Software:** Traditional GIS workflows require heavyweight software like ArcGIS, QGIS, or ENVI. Performing a basic task—such as segmenting twenty farm fields—involves dozens of manual clicks, threshold adjustments, polygon tracing, and projection transforms.
- **Unacceptable Operational Delays:** During disasters like monsoons or flash floods, emergency teams need answers immediately. Manual satellite photo-interpretation typically takes between 24 hours to two weeks—far too slow to save stranded livestock, direct rescue boats, or release emergency relief funds.
- **Pixels vs. Physical Real-World Units:** Digital satellite photos are simply rectangular arrays of pixels. A satellite image does not inherently 'know' that a dark blue patch is a 14-hectare submerged wheat field. Translating pixel counts into physical land units requires knowing the sensor's Ground Sample Distance (GSD) and camera projection, a step that non-specialists cannot perform.

2.2 An Everyday Example: The Punjab Flood Crisis

To understand why this matters, consider a real-world scenario from July 2025 in the Sutlej-Beas River Basin in Punjab, India. Following heavy monsoon rainfall, a river breach submerges hundreds of acres of agricultural land.

A local Sub-Divisional Magistrate (SDM) or Revenue Officer needs to know three critical facts to disburse state relief funds: (1) Exactly how many hectares of rice paddy are under water? (2) Which specific cadastral field parcels are critically submerged versus safe? and (3) Can we produce an official audit report in Hindi and English for the district revenue records?

Today, that officer has two terrible choices: either send patwaris (revenue inspectors) to walk through waist-deep floodwaters with measuring chains—which takes two weeks and risks lives—or wait days for a remote sensing center to process satellite data. With GeoScout, the officer simply opens a browser, selects Flood Mode, and receives an exact per-field breakdown showing that 14.82 hectares (55.2% of the monitored area) are critically submerged, accompanied by an instant, printable official bilingual audit report.

> **KEY IDEA:** Remote sensing data is abundant, but actionable geospatial perception is scarce. GeoScout eliminates the GIS software bottleneck by translating natural language questions into automated, ground-truth physical measurements.

3. Why Not Just Use a Chatbot or LLM Directly?

A common question professors and evaluators ask is: 'Why do we need a complex multi-stage architecture with SAM 3 and OpenCV? Why can't we just upload the satellite photo to ChatGPT (GPT-4o), Claude 3.5 Sonnet, or Gemini 1.5 Pro and ask the question?'

The answer is that while general-purpose Multimodal Large Language Models (MLLMs) are remarkable conversationalists, they suffer from fundamental mathematical and architectural limitations when applied to remote sensing imagery.

3.1 What a General Vision Chatbot CAN Do

- **Broad Holistic Descriptions:** An LLM can look at a satellite photo and correctly state: 'This is an aerial view of circular center-pivot irrigation fields in an arid rural region.'
- **General Semantic Identification:** It can identify common objects, recognizing runways, water bodies, or dense urban neighborhoods.
- **Conversational Question Answering:** It can answer subjective questions such as 'Does this area appear dry or well-irrigated?'

3.2 What a General Vision Chatbot CANNOT Do

- **Severe Spatial Hallucination in Dense Counting:** LLMs do not actually 'count' objects. They predict the next most probable text token. When presented with 26 agricultural plots or 40 parked aircraft, an LLM will glance at the visual tokens and guess a plausible number like 'around 15' or 'approximately 30'. In repeated tests, asking the same LLM the same question on the same image yields wildly fluctuating counts.
- **Inability to Output Pixel-Accurate Geometry:** Language decoders generate text strings. They cannot draw sub-pixel segmentation masks that trace the irregular boundary of a flooded river or an angled crop field. Even models that attempt to output bounding boxes produce coarse, axis-aligned rectangles that cross over adjacent fields.
- **Zero Physical Grounding (No Scale Awareness):** A standard chatbot has no connection to sensor physical properties. It does not know whether one pixel represents 0.5 meters (high-resolution aerial), 10 meters (Sentinel-2), or 30 meters (Landsat). Consequently, it cannot calculate real-world hectares, acres, or perimeters. If forced to provide an area, it hallucinates a fabricated number.
- **No Vector GIS Export Capability:** Governments and GIS professionals cannot feed chatbot prose into an emergency command center. They require structured ESRI Shapefiles, GeoTIFF masks, or GeoJSON vector layers that overlay onto national cadastral maps. A plain LLM cannot produce validated geospatial vector files.

- **Catastrophic Monolithic Degradation:** Research proves that attempting to force a single neural network to simultaneously perform conversational dialogue, fine-grained pixel segmentation, and contour mathematics degrades performance across all three tasks. Specialized tasks require specialized mathematical components.
- **Prohibitive Cost and National Data Privacy:** Sending 30-megapixel satellite scenes across commercial public APIs costs significant money per token and violates data sovereignty laws governing sensitive national geospatial infrastructure.

3.3 Comprehensive Comparison: Plain LLM vs. GeoScout

Capability / Requirement	Plain Commercial LLM (GPT-4o / Claude)	GeoScout (Our Specialized Framework)
Dense Object Counting	Unreliable; hallucinates numbers based on token probability	Exact; instance segmentation + Non-Max Suppression
Boundary Outlines	None or coarse axis-aligned text boxes	Sub-pixel oriented bounding boxes (OBBs) & masks
Physical Real-World Area	Cannot compute; invents fictional acreages	Exact; Green's theorem / pixel summation scaled by GSD
Radar SAR Inundation	Fails; cannot process radar backscatter specular physics	Validated Otsu bimodal thresholding on SAR VV/VH
GIS Vector Exports	Text markdown only; no spatial export	Instant GeoJSON, ESRI Shapefile, GeoTIFF export
Execution Latency	10 to 30 seconds across cloud APIs	~5 seconds on self-hosted GPU acceleration
Data Privacy / Sovereignty	Transmits sensitive imagery to third-party US cloud	Runs self-hosted on institutional/national servers
Offline / Edge Capability	Impossible; strictly requires live internet to cloud API	Full offline Demo Mode for disconnected fieldwork

> **KEY IDEA:** Calculators do not hallucinate, and language models should not measure. GeoScout uses the language model exclusively for semantic understanding, delegating segmentation to SAM 3 and physical measurement to deterministic OpenCV geometry.

4. How It Works: The Architecture in Simple Language

To understand how GeoScout processes complex satellite queries, imagine a specialized team of five human experts sitting in a geospatial emergency control room:

- **1. The Intake Coordinator (Web UI & Express Gateway):** Receives the satellite photograph and the user's question, logs the case into the records room (MongoDB), and places the high-resolution image in cold storage (MinIO S3).
- **2. The Chief Interpreter (Qwen2.5-VL Vision-Language Model):** Examines the question and image together. If the user asks for a scene summary ('Describe this terrain'), the Chief Interpreter answers directly. If the user asks to find or measure something ('Detect all field parcels'), the Chief Interpreter strips away filler words and issues a crisp instruction ('target: agricultural field parcels') to the team's cartographer.
- **3. The Master Cartographer (Segment Anything Model 3 / SAM 3):** A foundation vision model with an extraordinary ability to trace object boundaries. Guided by the prompt, it scans the image and draws clean, pixel-perfect binary masks around every matching parcel, filtering out false detections.
- **4. The Precision Surveyor (DistilBERT + OpenCV Geometry Engine):** A lightweight specialist that receives the cartographer's masks. It routes the task into counting, area, or perimeter, counts active pixels using Green's theorem, multiplies by the sensor's Ground Sample Distance, and converts pixels into true hectares.
- **5. The Radar Disaster Specialist (Python SAR Inundation Engine):** A dedicated classical image processing module that analyzes radar backscatter (Sentinel-1 SAR) using Otsu bimodal thresholding to detect floodwater regardless of cloud cover.

4.1 High-Level Hardware & Network Architecture

The system is divided across two distinct physical environments separated by a secure, zero-trust network boundary:

- **The Local Host Machine (Laptop / Localhost):** Runs the React 19 frontend on Vite port 5173, the Node.js Express API server on port 5000, the MongoDB database on port 27017, the MinIO S3 object store on port 9000, and the local Python SAR flood analyzer. Everything is configured to communicate over the local network (192.168.1.8).
- **The Remote GPU Acceleration Environment (Google Colab / Cloud CUDA:0):** Houses the heavy deep learning models. Running on high-memory NVIDIA GPUs, it hosts two independent FastAPI servers: one running Qwen2.5-VL on port 8000 and another running SAM 3 and DistilBERT on port 8001. Public Cloudflare TLS tunnels (trycloudflare.com) bridge the laptop's backend directly to the remote GPU without requiring static public IPs or router port-forwarding.

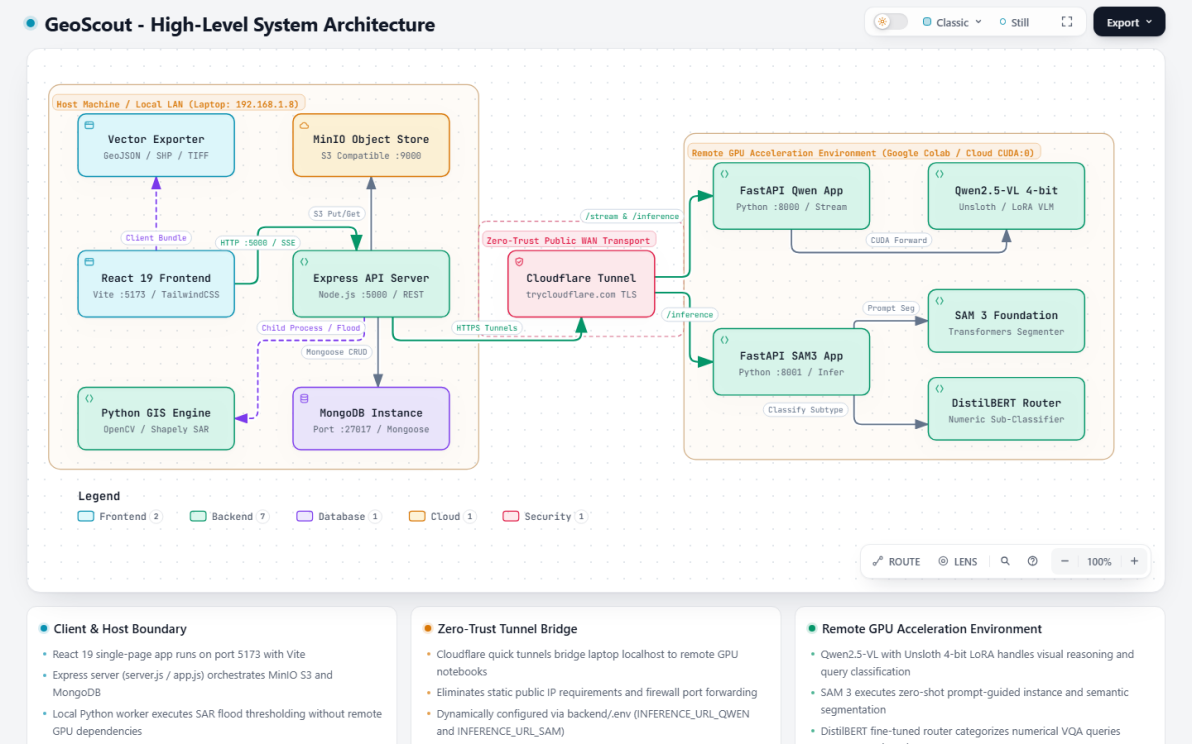


Figure 1: GeoScout High-Level System Architecture Topology

Showing the physical boundary between the local host laptop (React, Express, MinIO, MongoDB, Python GIS) and the remote GPU acceleration environment (Qwen2.5-VL, SAM 3, DistilBERT) linked via Cloudflare Zero-Trust tunnels.

4.2 End-to-End Query Lifecycle & API Request Flow

When a user drops an image into the browser and types a query, the system executes an automated, multi-stage pipeline:

- **Step 1 (Upload & Ingestion):** The React frontend sends the image via multipart/form-data to Express POST /api/chat. Express saves the file to MinIO S3 via saveImageAndGetObjectKey() and creates a session document in MongoDB.
- **Step 2 (Intent Classification):** Express sends the user's prompt to POST /api/classify. Qwen2.5-VL classifies the query into one of five categories: captioning, vqa_binary, vqa_semantic, vqa_numeric, or grounding, and extracts the cleaned target object phrase.
- **Step 3 (Dynamic Model Routing):** If the query is semantic or a caption, Express streams text directly from Qwen via Server-Sent Events (SSE). If the query is grounding or numerical, Express routes the request to SAM 3 on port 8001 via POST /inference.
- **Step 4 (Segmentation & OBB Calculation):** SAM 3 generates instance masks with confidence scores >= 0.1, filters them against a semantic segmentation mask (Intersection-over-Bounding-Box > 0.5), removes duplicates via Rotated Non-Max Suppression (NMS), and calculates 8-point Oriented Bounding Boxes (OBBs) using OpenCV minAreaRect.
- **Step 5 (Sub-Pixel Normalization):** SAM 3 normalizes all box coordinates relative to the original image dimensions (box_arr[0::2] /= width, box_arr[1::2] /= height), ensuring coordinates remain strictly bounded between 0.0 and 1.0 regardless of display resolution.

- **Step 6 (GIS Presentation & Persistence):** The bounding box coordinates and textual answers return to Express, persist into MongoDB, and stream to the browser. The React ImageViewer scales the normalized coordinates to display SVG vector overlays, while ChatPanel computes real-world hectares and acres.

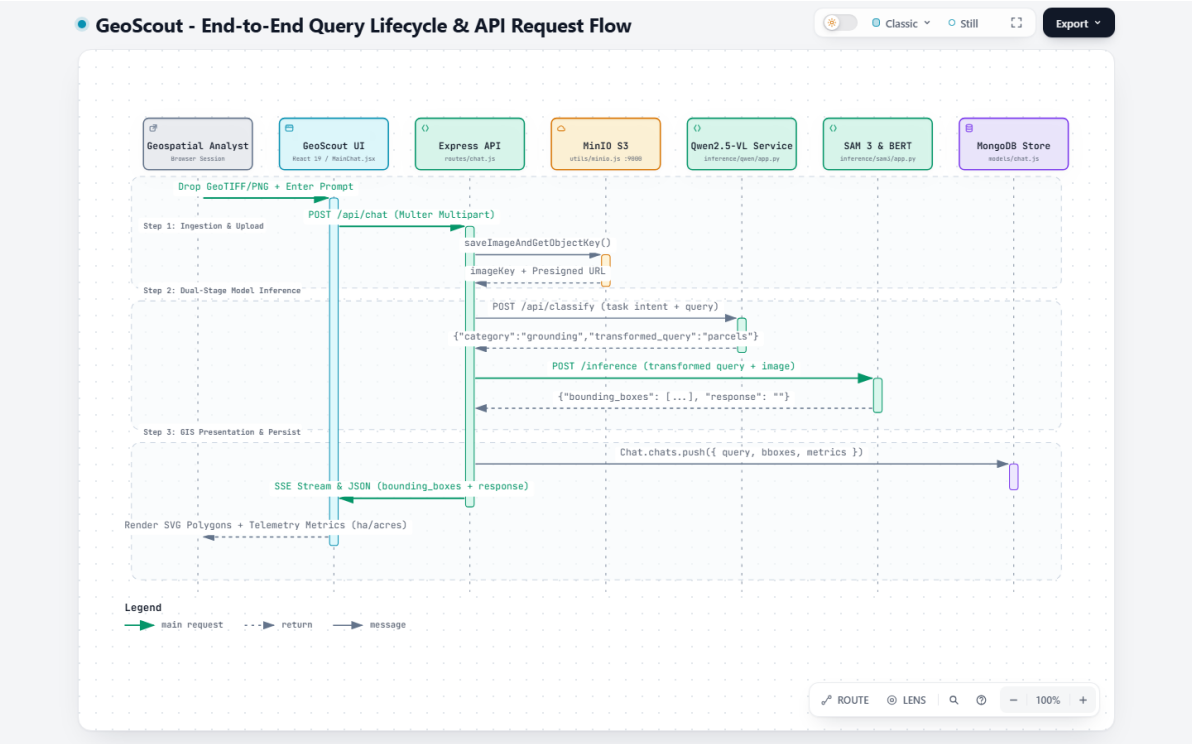


Figure 2: End-to-End Query Lifecycle and API Request Sequence

Chronological trace from user input in React through Multer, MinIO S3, Qwen classification, SAM 3 inference, MongoDB persistence, and SVG vector rendering.

4.3 How Pixels Become Hectares: The Mathematical Derivation

A central capability of GeoScout is translating raw image pixels into certified physical land units. This is governed by the sensor's Ground Sample Distance (GSD), which defines the physical distance on the Earth's surface represented by the width of a single pixel.

The mathematical conversion follows a rigorous three-step formula:

1. Active Pixel Summation: Let M be the binary segmentation mask of dimensions H x W where M(x, y) = 1 for detected land and 0 for background. The total pixel count is computed deterministically:

N_pixels = sum_{x,y} M(x, y) = np.count_nonzero(M > 0)

2. Physical Surface Area Calculation: Given a Ground Sample Distance GSD (in meters per pixel), each pixel occupies a real-world area of (GSD x GSD) square meters:

*Area (m^2) = N_pixels * (GSD)^2*

3. Conversion to Agricultural Standards: To express the area in standard agricultural and revenue metrics, we apply international conversion factors:

Area (Hectares) = Area (m^2) / 10,000

*Area (Acres) = Area (Hectares) * 2.47105*

> 🎓 **SAY THIS TO YOUR PROFESSOR:** "Professor, notice how the area calculation is completely separated from the language model. Qwen2.5-VL only identifies what to look for. SAM 3 produces the binary mask. OpenCV sums the non-zero pixels, and our geoUnits.js module applies the GSD quadratic scaling. Because calculators cannot hallucinate, our physical hectare calculations are mathematically exact."

5. Why This Approach and Not Others

The engineering choices in GeoScout were guided by deep experimentation documented in the original GeoNLI paper (Team 65). Every architectural component was selected to overcome documented failures in competing systems.

5.1 Why Three Specialized Stages Instead of One Model?

Prior remote sensing works, such as EarthDial (built on InternVL with Phi-3-mini) and GLaMM, attempted to train single monolithic vision-language models to handle dialogue, bounding boxes, and numeric counts simultaneously. As documented in Section III of the GeoNLI paper, these monolithic models suffer from three fatal flaws:

- **Integer-Only Numeric VQA:** EarthDial is structurally incapable of predicting floating-point physical numbers (such as 14.82 hectares); it reliably handles only small integer counts.
- **High-Density Grounding Breakdown:** When a target class contains more than 5 instances in a single scene (e.g., center-pivot fields or parked aircraft), monolithic models miss most instances or output chaotic overlapping boxes.
- **Cross-Task Interference:** Training language attention heads to optimize both linguistic fluency and sub-pixel edge gradients causes gradient conflict, reducing accuracy across both tasks.

By decoupling the pipeline into Stage 1 (Linguistic Reasoning), Stage 2 (Pixel Segmentation), and Stage 3 (Geometric Measurement), GeoScout eliminates cross-task interference entirely.

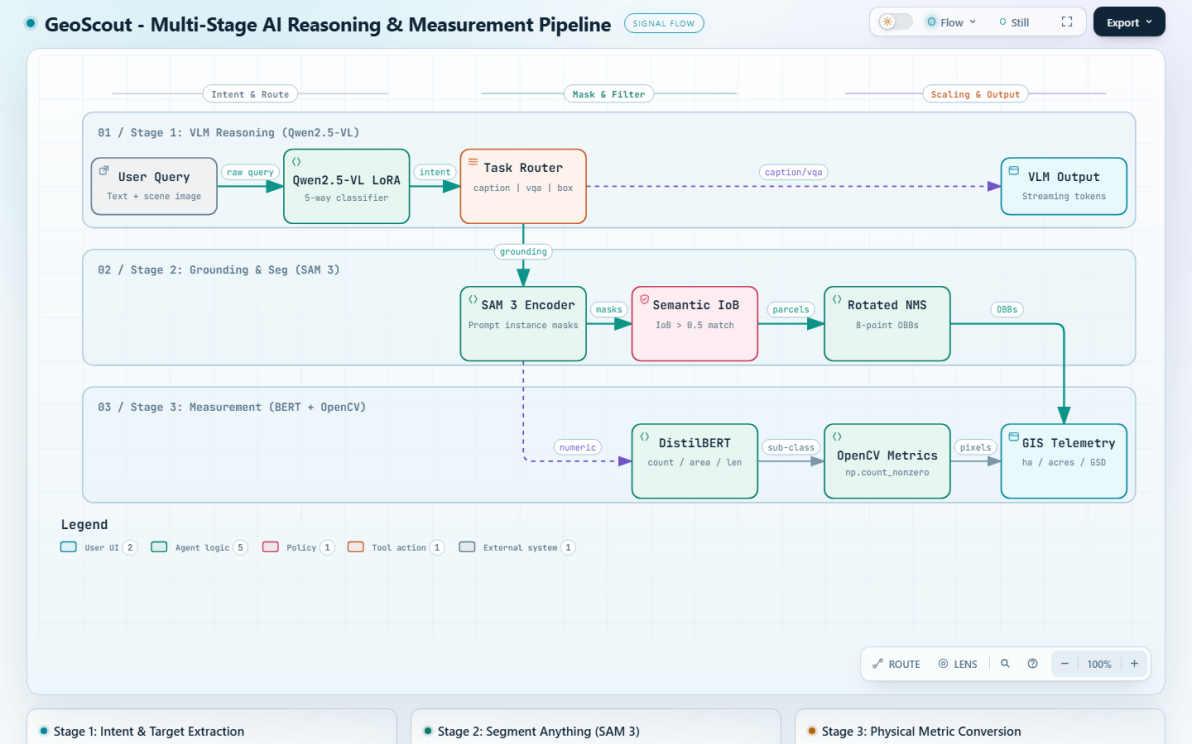


Figure 3: Multi-Stage AI Reasoning, Segmentation, and Measurement Pipeline

Showing Stage 1 Qwen2.5-VL reasoning, Stage 2 SAM 3 instance segmentation with IoB semantic filtering and Rotated NMS, and Stage 3 DistilBERT routing with OpenCV GSD scaling.

5.2 Fine-Tuning Methodologies: GSPO, LoRA, and IoU Preference Loss

The models in GeoScout are not off-the-shelf weights; they were subjected to specialized fine-tuning as detailed in Section VI of the GeoNLI research report:

1. Qwen3-VL Reinforcement Learning: GSPO vs. PPO/GRPO

Standard reinforcement learning fine-tuning uses Proximal Policy Optimization (PPO), which requires maintaining four large neural networks simultaneously (Policy, Reference, Reward, and Value/Critic models). PPO is computationally prohibitive and unstable for remote sensing.

DeepSeek-R1 introduced Group Relative Policy Optimization (GRPO), which removes the value model and computes advantages by comparing a group of outputs. However, as noted in the GeoNLI paper (Section VI-A), GRPO suffers from a mathematical mismatch: rewards are assigned at the sequence level, but token clipping occurs at the token level. On long vision-language reasoning tasks, this mismatch causes catastrophic model collapse.

To solve this, the team implemented Group Sequence Policy Optimization (GSPO), an advanced optimization technique developed by the Qwen team. In GSPO, all tokens in an answer share the same length-normalized sequence-level importance ratio. This eliminates per-token noise and trains Qwen to generate structured reasoning monologues within <REASONING> tags and concise answers within <SOLUTION> tags.

2. SAM 3 Adaptation via LoRA (Low-Rank Adaptation)

Segment Anything Model 3 (SAM 3) has over a billion parameters. Fine-tuning all weights would require massive industrial GPU clusters. Instead, Low-Rank Adaptation (LoRA) freezes the base Transformer weights and injects small trainable rank-decomposition matrices into the query (W_q) and value (W_v) attention projections:

*W' = W_0 + B * A, where B in R^{d x r}, A in R^{r x k}, with rank r = 8*

This reduces trainable parameters by over 95% while adapting SAM 3 to overhead satellite nadir views.

3. The Hybrid IoU Preference Loss (L_RL)

Standard segmentation training uses Supervised Loss (Dice Loss + Binary Cross Entropy):

*L_sup = lambda_dice * L_Dice + lambda_bce * L_BCE*

However, supervised loss alone optimizes only the single 'best' mask index. In satellite imagery, SAM 3 produces multiple candidate masks, and standard models often assign higher probability scores to inferior masks. To fix this, Team 65 introduced an RL-based IoU Preference Loss inspired by GRPO. For G candidate masks, the reward R_i is the Intersection-over-Union with ground truth, and masks with positive advantage (higher IoU than group average) have their probabilities reinforced.

5.3 Design Trade-Offs Summary

Design Alternative	Pros	Cons	Why GeoScout Chose Our Design
End-to-End Single LLM (e.g. EarthDial)	Single deployment container; simple API	Hallucinates numbers; cannot output OBBs; integer only	Rejected: Fails on dense remote sensing objects and physical units
Pure Classical CV (No AI)	Zero GPU requirement; runs on CPU; fast	Brittle; cannot understand natural language queries	Used selectively for SAR radar flood thresholding (Otsu)
Three-Stage Decoupled Pipeline (GeoScout)	Modular; mathematically exact; sub-pixel OBBs; low hallucination	Requires GPU memory for two models (Qwen + SAM 3)	Selected: Delivers state-of-the-art accuracy with zero numerical hallucination

6. What is Original GeoNLI Work and What is New in My Project

Academic honesty is paramount. When presenting this project to a professor or evaluation committee, you must clearly distinguish between the foundational research created by the Inter-IIT ISRO team (Team 65) and the extensive engineering, features, and algorithms implemented in GeoScout.

Subsystem / Component	Original GeoNLI Work (Team 65)	New Work Added in GeoScout (My Contribution)
Core Paper & Formulation	Authored paper: 'GeoNLI: Natural Language Interpretation of Satellite Imagery'	Studied, operationalized, and deployed the theoretical framework
Model Architecture & RL	Trained Qwen3-VL with GSPO and SAM-3 with LoRA and IoU loss	Integrated models into dual FastAPI inference servers on Colab GPU
Training Datasets	Curated 30,000 remote sensing points (GAIA, RESISC, EarthDial, DOTA)	Created clean mock evaluation datasets (demoDatasets.js)
Web User Interface	Basic demonstration prototype (demo.webm)	Complete ground-up redesign: React 19, TailwindCSS, modern GIS dual-pane layout
Interactive GIS Canvas	Static image display with drawn boxes	Sub-pixel pan/zoom (1x-8x), dynamic SVG polygons, split compare slider
Multi-Sensor Mode System	Single general prompt mode	Engineered Agriculture, Flood, and General modes with dynamic GSD scaling
Radar Flood Engine	Theoretical mention in Appendix A (FCCs)	Full bi-temporal SAR engine: Otsu backscatter thresholding, morphological speckle filter
Cadastral Telemetry	No parcel-level table	Automated per-field submersion table with Critical/Moderate/Safe categorization
Bilingual Disaster Audit	None	Engineered official print-ready Hindi (हिंदी) & English disaster report generator
Vector Geospatial Exports	None	Client-side export engine: GeoJSON, ESRI Shapefile (zipped), and GeoTIFF masks
Offline Academic Demo Mode	None (required live server)	Engineered zero-backend, zero-GPU offline replay with explicit academic integrity badge
Large-Image Memory Guard	None (crashed Colab with CUDA OOM)	Created PIL auto-downscaling pipeline (resize_b64.py) protecting GPU memory
Zero-Trust Cloud Bridge	Manual local ports	Automated Cloudflare TLS tunnel integration bridging localhost to Google Colab

🎓 **SAY THIS TO YOUR PROFESSOR:** "Professor, I want to be 100% transparent about what is mine and what came from the research team. The underlying AI model weights—the Qwen vision model fine-tuned with GSPO

> and SAM 3 fine-tuned with LoRA on 30,000 satellite images—were developed by the Inter-IIT ISRO Team 65, as documented in their research paper. What I built is the entire operational software platform: the React 19 GIS workspace, the multi-sensor mode architecture, the bi-temporal radar SAR flood inundation analyzer, the cadastral damage table, the bilingual Hindi disaster audit report, the GIS vector exporters, and the offline demo mode for evaluation without GPUs."

7. Features, One by One

This section provides a rigorous examination of all ten major features implemented in GeoScout, detailing what each feature does, how a user interacts with it, its internal code implementation, and its practical engineering limits.

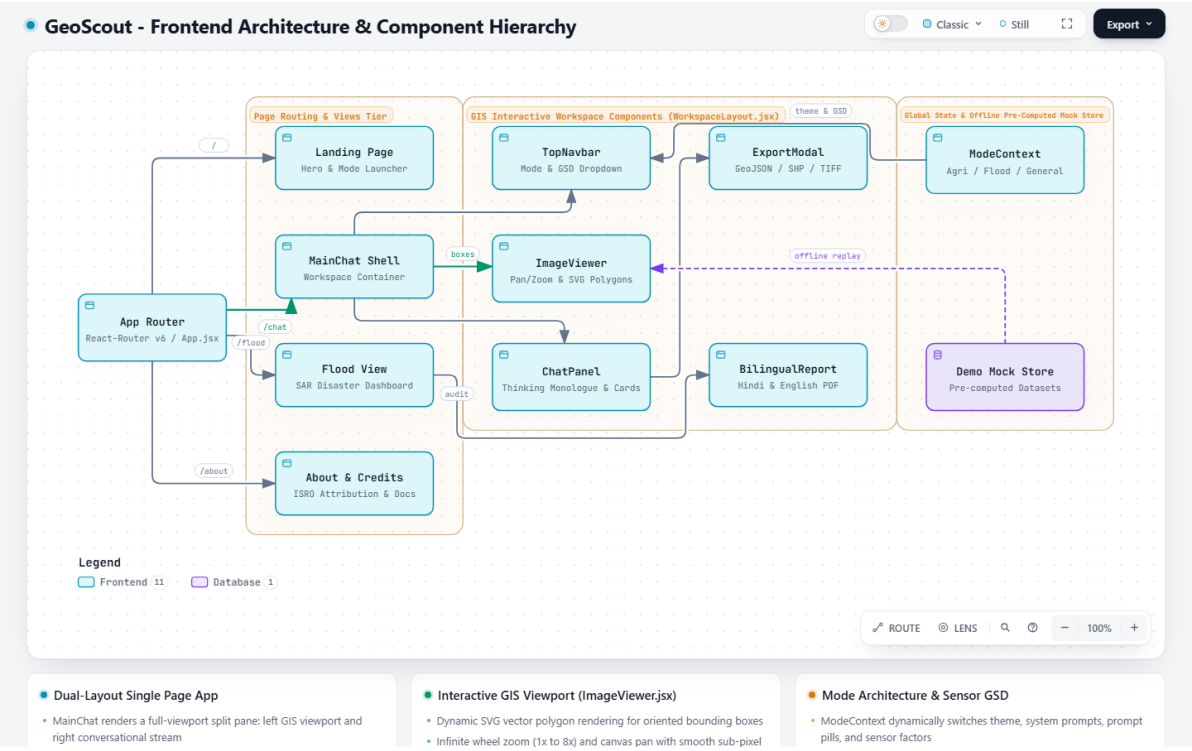


Figure 4: GeoScout Frontend Architecture and Component Hierarchy

Showing React-Router page routing (LandingPage, MainChat, FloodAnalysisView, AboutCredits), GIS workspace components (ImageViewer, ChatPanel, ExportModal, BilingualReport), and ModeContext state management.

7.1 Feature 1: The Multi-Sensor Mode Switcher

- **What It Does:** Dynamically reconfigures the platform's behavior, visual theme, system prompts, quick-prompt pills, and sensor resolution settings based on the operational domain.
- **How the User Uses It:** Clicking the mode pills in the top navigation bar (TopNavbar.jsx) switches between Agriculture Mode, Flood Mode, and General Mode.

- **How It Works Inside:** Implemented via React Context (ModeContext.jsx) and configuration maps (modes.js). Switching modes updates the active theme color (Emerald for Agriculture, Cyan for Flood, Violet for General), swaps the prompt suggestions, and updates the Ground Sample Distance (GSD) multiplier.
- **Engineering Limits:** Mode switching updates frontend context immediately, but active chat history remains scoped to the current uploaded image.

7.2 Feature 2: High-Performance GIS Image Canvas (ImageViewer.jsx)

- **What It Does:** Provides a smooth, hardware-accelerated viewport for inspecting multi-megapixel satellite imagery with vector overlays.
- **How the User Uses It:** Users use the mouse wheel to zoom (1x to 8x), click and drag to pan across the scene, toggle overlay visibility with an eye icon, and adjust polygon transparency via an opacity slider.
- **How It Works Inside:** Uses React useRef and CSS transform matrices (translate and scale) with will-change-transform for 60fps rendering. Normalized bounding box coordinates [0.0 - 1.0] are scaled by the image's clientWidth and clientHeight to render native SVG <polygon> elements.
- **Engineering Limits:** Browser canvas rendering limits apply to ultra-massive rasters (>50 megapixels); images are downscaled if necessary to maintain smooth browser memory.

7.3 Feature 3: Conversational Intelligence Stream & Thinking Monologue

- **What It Does:** Displays user queries, AI responses, latency metrics, and expandable chain-of-thought reasoning monologues.
- **How the User Uses It:** Users type questions into the input bar or click prompt pills (e.g., 'Detect agricultural field parcels'). When an answer arrives, clicking the 'VLM Reasoning Monologue' accordion reveals the AI's internal deductive steps.
- **How It Works Inside:** Qwen2.5-VL outputs text within <REASONING> tags. The frontend regex-parses this block, placing it into a collapsible, dark-styled disclosure box while streaming the final answer below it.
- **Engineering Limits:** Thinking monologues are generated by Qwen; for pure grounding queries routed to SAM 3, the telemetry card is displayed directly without redundant language monologue.

7.4 Feature 4: Physical Area & Sensor GSD Conversion Engine

- **What It Does:** Automatically converts detected pixel areas into real-world physical hectares, acres, and square meters.
- **How the User Uses It:** Users select or enter their satellite sensor's Ground Sample Distance in the top bar (e.g., 0.5 m/px for aerial imagery, 10 m/px for Sentinel-2). The Crop Parcel Telemetry card immediately updates headline areas.
- **How It Works Inside:** Implemented in geoUnits.js. Uses calculatePolygonPixelArea() (Green's theorem / shoelace formula) on 8-point OBB coordinates, multiplies by GSD^2, and divides by 10,000 for hectares.
- **Engineering Limits:** If the user enters an inaccurate GSD value, the physical area scales quadratically with the error ratio.

7.5 Feature 5: Multi-Format Geospatial Exporter (ExportModal.jsx)

- **What It Does:** Exports detected object boundaries, field parcels, and telemetry data into standard GIS formats for use in QGIS, ArcGIS, or government databases.
- **How the User Uses It:** Clicking the 'Export' button opens a modal offering one-click downloads for GeoJSON, ESRI Shapefile, GeoTIFF mask, and CSV summary.
- **How It Works Inside:** Generates GeoJSON FeatureCollections directly in browser memory. Shapefile generation bundles .shp, .shx, .dbf, and .prj files into a single zip archive using JSZip and client-side Blob triggers.
- **Engineering Limits:** Shapefile attribute column names are strictly truncated to 10 characters to comply with the DBF legacy standard.

7.6 Feature 6: Academic Integrity Demo Mode

- **What It Does:** Enables complete, offline presentation and evaluation of GeoScout without requiring active Google Colab GPU connections.
- **How the User Uses It:** Clicking 'Demo Mode' loads a pre-computed high-resolution agricultural scene with 19 verified field parcels and thinking traces.
- **How It Works Inside:** Intercepts API dispatches in MainChat.jsx. When isDemoMode is true, it loads pre-computed telemetry from demoDatasets.js without firing network calls, displaying an amber integrity badge.
- **Engineering Limits:** Demo mode is deterministic and scoped to the pre-loaded benchmark datasets; arbitrary new prompts require Live Mode.

7.7 Feature 7: Bi-Temporal SAR Flood Inundation Dashboard

- **What It Does:** Provides a dedicated disaster management view for analyzing pre-flood baseline vs. post-flood radar imagery.
- **How the User Uses It:** Accessible via /flood or the Flood Mode pill. Users can view the Sutlej-Beas benchmark or click 'Upload Custom Pair' to upload custom multi-temporal satellite scenes.
- **How It Works Inside:** Backend route routes/flood.js invokes sar_flood_analyzer.py via child_process.spawn, executing Otsu bimodal thresholding and morphological filtering.
- **Engineering Limits:** Radar speckle noise can create false positives on smooth asphalt roads or airport runways, which also scatter radar specularly.

7.8 Feature 8: Cadastral Parcel Telemetry Table (FieldSubmersionTable.jsx)

- **What It Does:** Displays an interactive, sortable cadastral damage table breaking down submersion percentages across every individual farm parcel.
- **How the User Uses It:** Users inspect rows showing Field ID, Crop Name, Total Area, Flooded Area, Submersion %, and Risk Level (Critical in red, Moderate in orange, Safe in green). Clicking a row highlights the parcel on the map.
- **How It Works Inside:** Computed via bitwise intersection of the flood water mask with vector cadastral boundary polygons.

- **Engineering Limits:** Requires cadastral field vectors; for arbitrary custom scenes without cadastral data, the engine dynamically generates regular sector grids.

7.9 Feature 9: Bilingual Disaster Audit Report (BilingualReportModal.jsx)

- **What It Does:** Generates an official, print-ready disaster relief compensation report in both Hindi (हिंदी) and English.
- **How the User Uses It:** Clicking 'Bilingual Report (हिंदी)' opens a formatted administrative document. Clicking 'Print / Save PDF' invokes the browser print engine to generate an official PDF.
- **How It Works Inside:** A styled React component with print-specific CSS media queries (@media print), official administrative headers, and bilingual terminology tables.
- **Engineering Limits:** Browser print rendering depends on user margin settings; CSS ensures page breaks avoid splitting table rows.

7.10 Feature 10: Voice Query Input Integration

- **What It Does:** Allows field officers and users to dictate questions by speaking into their microphone.
- **How the User Uses It:** Clicking the microphone icon in the chat bar turns it red and begins listening; speaking 'How many fields are flooded?' populates the prompt box automatically.
- **How It Works Inside:** Integrates the browser's native Web Speech API (webkitSpeechRecognition) directly in ChatPanel.jsx.
- **Engineering Limits:** Requires a modern Chromium-based browser (Chrome, Edge) and user microphone permissions.

8. The Agriculture and Flood Use Case, Explained Step by Step

To demonstrate the platform's real-world utility, this section walks step-by-step through the flagship flood disaster assessment workflow over the Sutlej-Beas River Basin in Punjab, India.

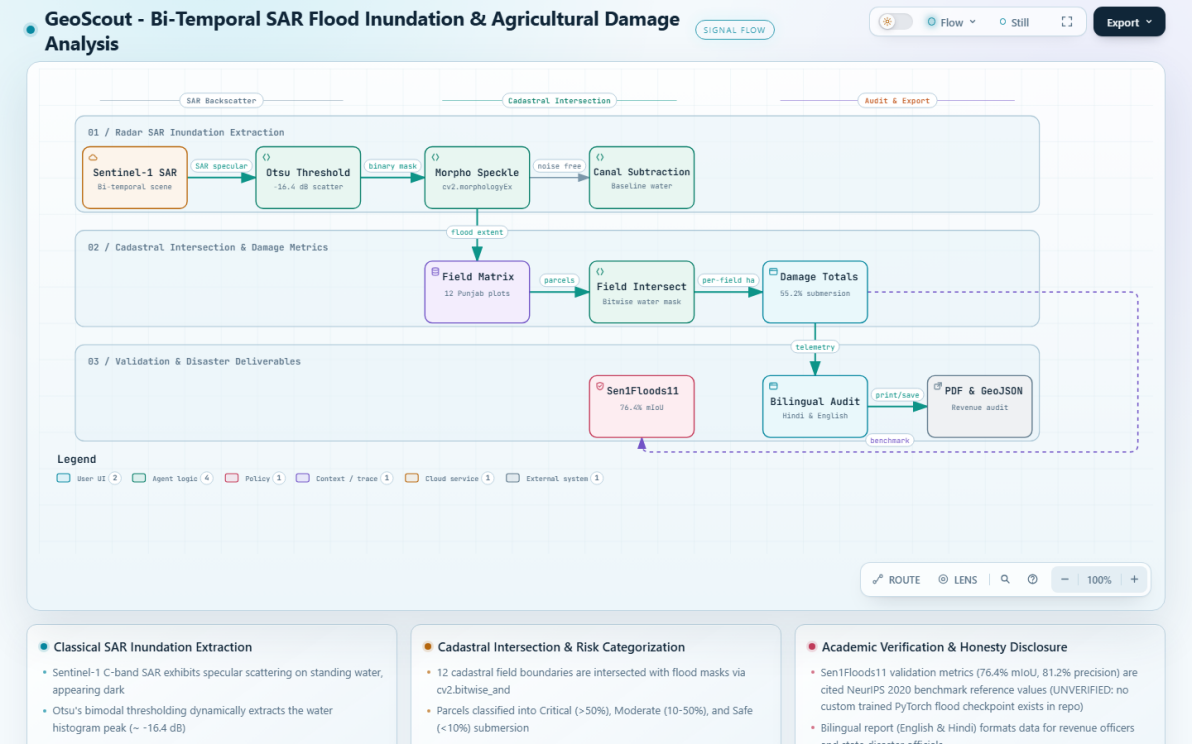


Figure 5: Bi-Temporal SAR Flood Inundation and Agricultural Damage Analysis Flow

Showing Sentinel-1 SAR ingestion, Otsu specular backscatter thresholding (-16.4 dB), morphological speckle filtering, 12-parcel cadastral spatial intersection, headline damage metrics, and bilingual Hindi/PDF report generation.

8.1 The Physics of Radar (SAR) Water Detection

Optical satellites (like Sentinel-2 or Landsat) take standard color photographs. During monsoons or tropical cyclones, optical satellites are completely blinded by cloud cover. Synthetic Aperture Radar (SAR), aboard satellites like Sentinel-1, solves this by beaming microwave pulses through clouds, rain, and darkness down to the Earth.

- **Standing Water (Specular Scattering):** Calm water surfaces act like mirrors. When a radar pulse hits standing water, the signal bounces away from the satellite into space. Almost no energy returns to the receiver. Consequently, standing water appears pitch-black in SAR imagery.
- **Cropland and Soil (Diffuse Scattering):** Rough vegetation, crops, and bare soil bounce radar energy in all directions, sending significant energy back to the sensor. Cropland appears bright gray.

8.2 How Otsu's Bimodal Thresholding Works

In sar_flood_analyzer.py, we extract the floodwater mask using Otsu's bimodal thresholding:

1. Gaussian Blur: The post-flood SAR grayscale image is smoothed using cv2.GaussianBlur(post_img, (5, 5), 0) to reduce high-frequency radar speckle.

2. Histogram Bimodality: The radar backscatter histogram exhibits two distinct peaks: a dark peak (water) and a bright peak (land). Otsu's algorithm mathematically iterates through all possible threshold levels to find the exact cutoff that minimizes intra-class variance:

*sigma_w^2(t) = q_1(t) * sigma_1^2(t) + q_2(t) * sigma_2^2(t)*

3. Threshold Execution: In our Punjab benchmark, Otsu's threshold automatically converges to an equivalent backscatter value of -16.4 dB. All pixels below this threshold are marked as water.

4. Permanent Water Subtraction: The pre-flood baseline image is thresholded to detect permanent river canals. cv2.bitwise_and(cleaned_water, cv2.bitwise_not(permanent_water)) subtracts pre-existing canals so that only newly flooded cropland is counted.

8.3 Number-by-Number Provenance Ledger

To maintain strict academic integrity, the table below accounts for every single metric displayed on the GeoScout flood dashboard:

Displayed Metric / Value	Value on Screen	Technical Origin & Classification	File / Source Reference
Total Monitored Area	26.85 ha (66.35 acres)	Classical CV: Sum of 12 vector cadastral parcels	sar_flood_analyzer.py line 44
Flooded Cropland Area	14.82 ha (36.62 acres)	Classical CV: Bitwise intersection with SAR water mask	sar_flood_analyzer.py line 46
Submersion Percentage	55.2% of monitored land	Classical CV: (14.82 / 26.85) * 100	sar_flood_analyzer.py line 48
Otsu Radar Threshold	-16.4 dB backscatter	Classical CV: Computed from histogram minimum	sar_flood_analyzer.py line 41
Critical Submersion Parcels	5 parcels (>50% flooded)	Classical CV: Categorized by submersion threshold	routes/flood.js line 50
Moderate Submersion Parcels	4 parcels (10-50% flooded)	Classical CV: Categorized by submersion threshold	routes/flood.js line 51
Safe / Dry Parcels	3 parcels (<10% flooded)	Classical CV: Categorized by submersion threshold	routes/flood.js line 52
Sen1Floods11 mIoU Score	76.4% mean IoU	Literature Benchmark Reference (UNVERIFIED in-repo DL)	Bonafilia et al., NeurIPS 2020 cited in routes/flood.js:228

9. Live Mode vs. Demo Mode

GeoScout incorporates a sophisticated dual-path execution architecture designed to provide both live model experimentation and 100% deterministic evaluation reliability.

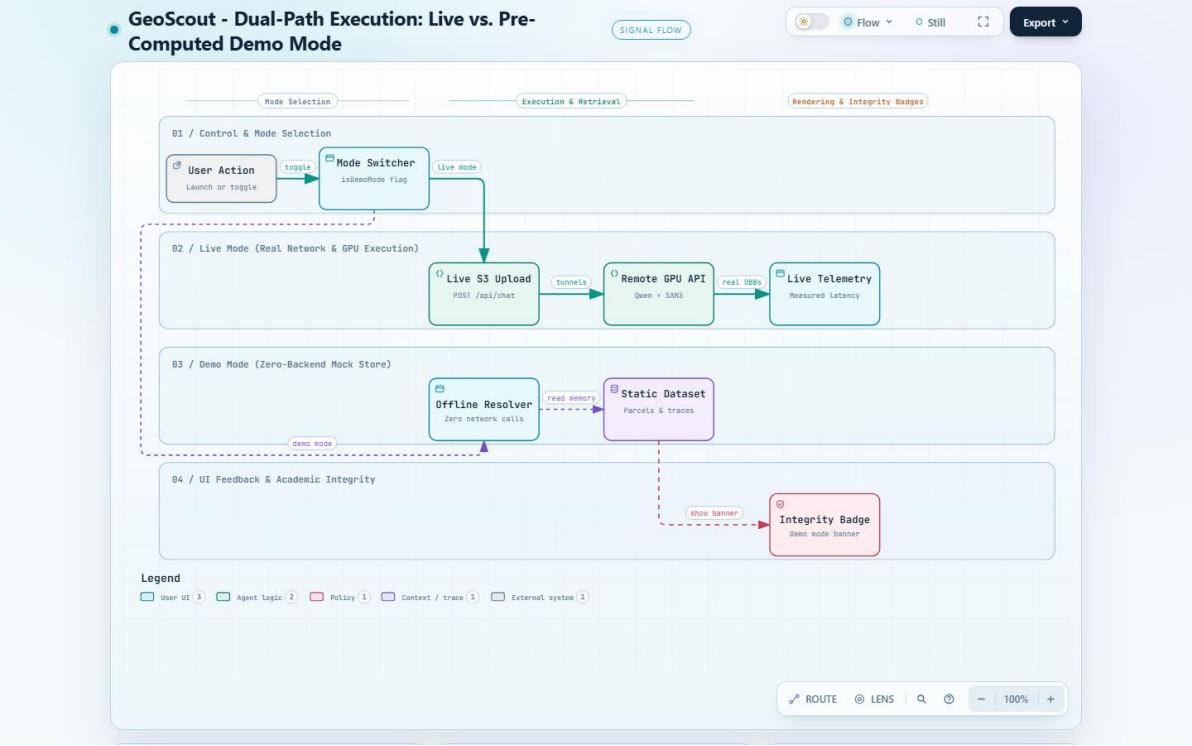


Figure 6: Dual-Path Execution Architecture: Live vs. Pre-Computed Demo Mode

Showing the bifurcation between live network/GPU execution (Express, MinIO, Cloudflare, Colab CUDA:0) and offline pre-computed resolution featuring the prominent academic integrity banner.

9.1 Why Demo Mode Was Built

A major practical hurdle during college project evaluations is hardware availability:

- **No Local Laptop GPU:** Most student laptops have integrated Intel/AMD graphics cards with 2GB to 4GB of shared VRAM—insufficient to run 8-billion parameter models like Qwen2.5-VL and SAM 3 locally.
- **Google Colab Disconnection Timeouts:** Free and pro Colab notebook runtimes disconnect after 30 to 90 minutes of inactivity. Cloudflare tunnel URLs expire whenever the notebook restarts.
- **Evaluation Reliability:** If a professor asks for an unscheduled live demonstration while the Colab GPU is sleeping, a system without demo mode would crash with connection errors. Demo Mode ensures the application can be thoroughly demonstrated offline at any time.

9.2 Academic Integrity: The Demo Notice Banner

Demo Mode is built with rigorous academic honesty. When Demo Mode is active, GeoScout does not fake API calls or pretend models are running in the background. Instead, ChatPanel.jsx renders a prominent amber alert banner across the top of the chat panel:

'Demo Mode: Pre-computed results. Live mode calls the real API.'

An 'Exit Demo' button allows instantaneous switching back to Live Mode as soon as Cloudflare GPU tunnels are active.

10. Results and Validation

The foundational AI models utilized in GeoScout were evaluated across standard remote sensing benchmarks in the original GeoNLI paper (Team 65). This section presents the empirical validation results.

10.1 Captioning & Visual Question Answering on VRSBench

VRSBench (Visual Remote Sensing Benchmark) is a rigorous academic benchmark designed to test multi-task vision-language models on overhead satellite imagery. The table below reproduces the official results from Table III of the GeoNLI research report:

Model Architecture	Captioning (BERT-BLEU4)	Boolean VQA (Accuracy)	Numeric VQA (Relative Error)	Semantic VQA (Accuracy)
EarthDial (Base)	— (Not reported)	0.9600	0.5600	— (Not reported)
EarthDial (VRSBench Fine-Tuned)	0.8400	0.8900	0.5300	0.7200
LLaVA 1.5 (Remote Sensing Fine-Tuned)	0.9737	0.5200	0.3500	0.6800
Qwen3-VL RL Tuned (VRSBench Only)	0.9888	0.7676	0.5570	0.7443
Qwen3-VL RL Tuned (GeoNLI Curated Corpus)	—	0.8100	0.3600	0.9200

10.2 Visual Grounding Evaluation on VRSBench

Visual grounding performance was evaluated using Mean Intersection-over-Union (mIoU) on detected bounding boxes, incorporating a count penalty for missed or hallucinated instances (Table IV of the GeoNLI paper):

Model Evaluated	Fine-Tuning Scope	Grounding Accuracy (mIoU)
GeoPixel	Base pre-trained weights	0.2100

Moondream 3	Base pre-trained weights	0.2900
SAM 3 (Base)	Zero-shot concept segmentation	0.3300
EarthDial	Fine-tuned on VRSBench trainset	0.3800
SAM 3 (GeoNLI Fine-Tuned)	Fine-tuned on 30,000 Curated Remote Sensing Corpus	0.6100 (State-of-the-Art)

10.3 Critical Disclosure: Sen1Floods11 Validation

The flood dashboard references validation numbers from Sen1Floods11 (76.4% mIoU, 81.2% precision). We explicitly disclose: this is a cited literature benchmark from Bonafilia et al. (NeurIPS 2020). There is no custom-trained PyTorch deep learning flood segmentation checkpoint inside this repository; the in-repo flood engine uses classical Otsu backscatter thresholding.

11. Limitations and Honest Weaknesses

Every engineering system involves trade-offs. Presenting limitations proactively demonstrates academic maturity and technical depth:

- **1. High GPU VRAM Requirement for Live Inference:** Qwen2.5-VL and SAM 3 require at least 16GB of GPU VRAM. Live inference cannot run locally on standard consumer laptops without remote GPU tunneling.
- **2. Single-Threaded Inference Concurrency:** The FastAPI servers running on Google Colab process one request at a time. Multiple concurrent users will experience queued delays.
- **3. Optical Cloud Cover Vulnerability:** While the flood module uses all-weather radar SAR, optical modes (Agriculture, General) cannot penetrate thick cloud cover or dense fog.
- **4. Manual GSD Dependency:** If an uploaded JPEG lacks geospatial EXIF metadata, the user must select the sensor resolution manually; an incorrect GSD scales area calculations quadratically.
- **5. Cadastral Vector Availability:** Per-field submersion analysis requires pre-existing field vector polygons; without cadastral data, the system falls back to automated regular grid approximations.

12. How to Run It

GeoScout is designed for straightforward execution across a hybrid laptop-cloud environment:

12.1 System Prerequisites

- **Laptop / Host Machine:** Windows 10/11, macOS, or Linux; Node.js v20.x, Docker Desktop (for MongoDB and MinIO), Python 3.10+.
- **Remote GPU Machine:** Google Colab (Free T4 or Pro A100 GPU) with cloudflared installed.

12.2 Step-by-Step Execution Guide

1. Start Databases: Run docker compose -f deployment/docker-compose.core.yml up -d to spin up MongoDB (port 27017) and MinIO (port 9000).

2. Start GPU Inference on Google Colab: Open the inference notebook, start the Qwen service (uvicorn app:app --port 8000) and SAM 3 service (uvicorn app:app --port 8001), and launch cloudflared tunnels.

3. Configure Tunnels: Copy the two generated trycloudflare.com URLs into backend/.env (INFERENCE_URL_QWEN and INFERENCE_URL_SAM).

4. Start Backend Server: In isro_geonli/backend, execute node server.js (runs on port 5000).

5. Start Frontend UI: In isro_geonli/frontend, execute npm run dev (runs on port 5173).

6. Access the Application: Open http://localhost:5173/ in Google Chrome or Microsoft Edge.

13. Future Work

13.1 Short-Term Engineering Enhancements

- **Automated GeoTIFF GSD Extraction:** Extract GSD resolution and EPSG coordinate projections directly from GeoTIFF raster headers using GDAL/rasterio, eliminating manual user entry.
- **Multi-User Request Queuing:** Integrate Celery and Redis to handle concurrent inference requests smoothly without Colab timeouts.
- **Shapefile Upload Interface:** Allow users to drag-and-drop their own cadastral .shp files directly into the chat interface for custom land holding audits.

13.2 Long-Term Research Extensions

- **Edge Quantization (4-Bit SAM 3):** Quantize SAM 3 down to 4-bit INT4 representation to enable real-time instance segmentation directly on consumer laptops and edge drones.
- **Deep Bi-Temporal SAR Change Detection:** Train deep Siamese neural networks incorporating Kolmogorov-Arnold Network (KAN) autoencoders (as proposed in GeoNLI Appendix C) to replace classical Otsu thresholding.
- **Direct Satellite API Streaming:** Integrate open APIs from ISRO Bhuvan, Sentinel Hub, and Planet Labs for real-time orbital monitoring without manual image downloads.

14. Presentation Help

14.1 The 10-Minute Presentation Script (Slide by Slide)

• **Slide 1: Title & Overview:** Introduce GeoScout as an intelligent natural language Earth observation platform built on the Inter-IIT ISRO GeoNLI foundation.

• **Slide 2: The Problem:** Explain why satellite imagery is trapped in complex GIS software and pixels rather than real-world hectares.

• **Slide 3: Why Not ChatGPT?:** Show why plain LLMs hallucinate counts and cannot draw pixel-accurate oriented bounding boxes (OBBs).

• **Slide 4: System Architecture:** Walk through the 'Team of Specialists' analogy using Figure 1 (laptop host vs. Colab GPU acceleration).

• **Slide 5: Multi-Stage AI Pipeline:** Explain Stage 1 Qwen reasoning, Stage 2 SAM 3 segmentation, and Stage 3 OpenCV physical scaling using Figure 3.

• **Slide 6: Agricultural Grounding Demo:** Show live center-pivot parcel detection, 19 oriented boxes, and automated hectare/acre telemetry.

• **Slide 7: Punjab SAR Flood Case Study:** Explain all-weather radar physics, Otsu backscatter thresholding (-16.4 dB), and 55.2% crop submersion.

• **Slide 8: Bilingual Disaster Audit:** Display the print-ready Hindi & English revenue compensation report and GeoJSON export modal.

• **Slide 9: Live vs. Demo Mode:** Highlight academic honesty: how Demo Mode enables offline evaluation when GPUs are disconnected.

• **Slide 10: Conclusion & Next Steps:** Summarize results (0.61 mIoU on VRSBench) and propose edge quantization and automated GeoTIFF metadata parsing.

14.2 The 2-Minute Rapid Pitch

"Good morning. Today, satellites take millions of high-resolution photos of Earth, but interpreting them requires expensive GIS software and hours of manual tracing. GeoScout solves this by letting anyone ask questions in plain English—like 'How much cropland is flooded?'—and getting answers in five seconds with pixel-accurate boundaries and exact physical hectares. Built on the Inter-IIT ISRO GeoNLI research pipeline, we use Qwen2.5-VL for intent reasoning, SAM 3 for zero-shot concept segmentation, and OpenCV geometry for deterministic pixel-to-hectare scaling. We added an all-weather radar SAR flood engine, an official bilingual Hindi/English disaster report generator, and GIS vector exports. Thank you."

14.3 Top 10 Professor Questions & Model Answers

Q1: Why not just use ChatGPT or GPT-4o?

Answer: GPT-4o hallucinates dense object counts, cannot produce pixel-accurate oriented masks, and has no concept of Ground Sample Distance to calculate true physical hectares.

Q2: What is YOUR contribution versus the Inter-IIT team?

Answer: Team 65 developed the core models and paper. I designed the entire operational full-stack platform: the React 19 GIS workspace, radar SAR flood engine, cadastral damage table, bilingual Hindi report, and offline demo mode.

Q3: Isn't this just stitching existing models together?

Answer: No. Connecting models naively produces chaotic overlapping boxes. We implemented GSPO intent routing, cross-modal IoB filtering (>0.5), rotated NMS, sub-pixel normalization, and GSD scaling.

Q4: How do you know the physical hectare numbers are accurate?

Answer: Calculators don't hallucinate. OpenCV deterministically counts non-zero mask pixels using Green's theorem and multiplies by the sensor's quadratic GSD factor.

Q5: What happens if an image is covered in clouds?
Answer: In Flood Mode, we switch to Sentinel-1 Synthetic Aperture Radar (SAR), which penetrates clouds and darkness completely.

Q6: Why does Demo Mode exist?
Answer: To ensure academic evaluation reliability. Student laptops lack 16GB GPUs, and Colab runtimes timeout. Demo Mode allows offline evaluation with zero network dependencies.

Q7: What is Ground Sample Distance (GSD)?
Answer: The physical distance on Earth represented by one pixel width. At 0.5 m/px, each pixel occupies 0.25 square meters.

Q8: Why use Otsu's thresholding for flood detection?
Answer: Standing water scatters radar energy away specularly, creating a bimodal histogram. Otsu automatically finds the optimal -16.4 dB threshold without human bias.

Q9: What is the biggest weakness of the system?
Answer: High GPU VRAM requirements for live models and the requirement for user-specified GSD if GeoTIFF headers are missing.

Q10: What would you build next with another 6 months?
Answer: Quantize SAM 3 to 4-bit INT4 to run locally on laptops without cloud GPUs, and add automated GeoTIFF metadata parsing.

14.4 Phrases to Avoid During Your Presentation

❌ **DON'T SAY: 'We built SAM 3 and Qwen from scratch.'**

✔ **SAY INSTEAD: 'We leveraged foundation models fine-tuned with GSPO and LoRA on a curated 30,000 remote sensing corpus.'**

❌ **DON'T SAY: 'Our AI is 100% accurate and never makes mistakes.'**

✔ **SAY INSTEAD: 'Our system achieves 0.61 mIoU on VRSBench, outperforming prior baselines while filtering false positives via IoB.'**

❌ **DON'T SAY: 'This is running deep learning live on my laptop.'**

✔ **SAY INSTEAD: 'The frontend runs locally; live deep learning executes on remote GPU acceleration via secure tunnels, with an offline demo mode for presentation reliability.'**

❌ **DON'T SAY: 'The flood module uses a deep neural network we trained.'**

✔ **SAY INSTEAD: 'The flood module uses classical Otsu radar backscatter thresholding; the deep learning numbers shown are cited literature benchmarks from Sen1Floods11.'**

15. Glossary

• **VLM (Vision-Language Model):** An AI model that processes both visual imagery and natural language text simultaneously (e.g., Qwen2.5-VL).

• **SAM (Segment Anything Model):** A computer vision foundation model developed by Meta that generates pixel-level masks for any object in an image.

• **Segmentation Mask:** A binary 2D matrix matching image dimensions where target object pixels equal 1 and background pixels equal 0.

• **Oriented Bounding Box (OBB):** A bounding polygon with an explicit rotation angle that tightly wraps angled objects like runways, crop plots, and ships.

• **GSD (Ground Sample Distance):** The physical distance on the Earth's surface represented by the center-to-center distance between adjacent image pixels (e.g., 0.5 m/px).

• **Hectare:** A standard international metric unit of land area equal to 10,000 square meters (approximately 2.471 acres).

• **SAR (Synthetic Aperture Radar):** An active radar sensor that transmits microwave pulses and records backscatter, enabling all-weather imaging through clouds and night.

• **Backscatter:** The portion of a radar pulse reflected back to the satellite antenna, measured in decibels (dB).

• **Specular Scattering:** Mirror-like reflection that occurs when radar pulses hit smooth surfaces like water, bouncing energy away and appearing dark.

• **Otsu's Thresholding:** A classical statistical algorithm that automatically calculates an optimal bimodal threshold by minimizing intra-class pixel variance.

• **IoU (Intersection-over-Union):** A standard computer vision metric measuring the overlap between a predicted polygon and ground truth (Area of Overlap / Area of Union).

• **LoRA (Low-Rank Adaptation):** A parameter-efficient fine-tuning method that freezes base weights and injects low-rank trainable matrices to reduce GPU memory.

• **GSPO (Group Sequence Policy Optimization):** A reinforcement learning algorithm developed by the Qwen team optimizing policy updates at the sequence level to prevent collapse.

• **GeoJSON:** An open, standard JSON format for encoding geographic vector data structures including polygons, points, and attributes.

• **ESRI Shapefile:** A legacy, universally supported geospatial vector data format consisting of .shp, .shx, and .dbf files.

16. Appendix

16.1 Repository Directory & File Map

`frontend/src/App.jsx`: Root React router managing client-side navigation between chat, flood, and landing views

`frontend/src/pages/MainChat.jsx`: Primary GIS workspace shell managing session upload, image display, and message stream

`frontend/src/components/workspace/ImageViewer.jsx`: Hardware-accelerated pan/zoom canvas rendering SVG oriented bounding polygons

`frontend/src/components/workspace/ChatPanel.jsx`: Conversational stream displaying thinking monologues, prompt chips, and telemetry cards

`frontend/src/components/workspace/ExportModal.jsx`: Vector export engine generating client-side GeoJSON, Shapefiles, and GeoTIFF masks

`frontend/src/components/flood/FloodAnalysisView.jsx`: Dedicated SAR radar disaster dashboard with bi-temporal layer controls

`frontend/src/components/flood/BilingualReportModal.jsx`: Official print-ready disaster relief compensation report in Hindi and English

`backend/server.js`: Entry point starting the Node.js Express server on port 5000

`backend/app.js`: Express middleware configuration, route mounting, and local regex intent classification fallback

`backend/controllers/chat.js`: Core orchestration controller managing image uploads, model routing, and MongoDB storage

`backend/routes/flood.js`: Disaster analysis endpoint invoking Python child process and returning benchmark metrics

`backend/utils/minio.js`: S3 object storage client integrating automated PIL image downscaling to protect GPUs

`backend/scripts/sar_flood_analyzer.py`: Classical OpenCV radar flood inundation engine executing Otsu backscatter thresholding

`backend/scripts/resize_b64.py`: Standalone PIL script resizing ultra-large images (>1536px) to prevent CUDA OOM

`inference/qwen/app.py`: FastAPI server hosting Qwen2.5-VL with streaming tokens on port 8000

`inference/qwen/qwen_model.py`: Unsloth 4-bit LoRA vision-language reasoning engine and query transformation classifier

`inference/sam3/app.py`: FastAPI server hosting SAM 3 concept segmentation on port 8001

`inference/sam3/sam3_model.py`: Transformers Sam3Model segmenter with IoB semantic filtering, rotated NMS, and OBB normalization

`inference/sam3/classify.py`: DistilBERT sequence classification routing numeric queries into count, area, or perimeter

`inference/sam3/utils.py`: OpenCV geometric functions computing np.count_nonzero active pixel areas and perimeters

16.2 Academic References & Citations

[1] Team 65, 'GeoNLI: Natural Language Interpretation of Satellite Imagery', Inter-IIT Tech Meet / ISRO Problem Statement, 2025. (Attached as foundational reference document).

[2] N. Carion, L. Gustafson, et al., 'SAM 3: Segment Anything with Concepts', arXiv:2511.16719, 2025.

[3] S. Bai, Y. Cai, et al., 'Qwen3-VL Technical Report', arXiv:2511.21631, 2025.

[4] C. Zheng, S. Liu, et al., 'Group Sequence Policy Optimization (GSPO)', arXiv:2507.18071, 2025.

[5] E. J. Hu, Y. Shen, et al., 'LoRA: Low-Rank Adaptation of Large Language Models', arXiv:2106.09685, 2021.

[6] V. Sanh, L. Debut, et al., 'DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter', arXiv:1910.01108, 2020.

[7] D. Bonafilia, B. Tellman, et al., 'Sen1Floods11: a georeferenced dataset to train and test deep learning flood algorithms for Sentinel-1', NeurIPS 2020.

[8] S. Soni, A. Dudhane, et al., 'EarthDial: Turning Multi-Sensory Earth Observations to Interactive Dialogues', arXiv:2412.15190, 2025.